

Natural Language Processing (CS4063)

Assignment 1 [22 Marks]

Text Similarity and Word Representations Analysis

CLO2: represents documents and words as vectors for a variety of NLP tasks

Objective:

Create a text similarity engine to recommend content based on a user's input. The engine will leverage sparse (TF-IDF) and dense (word embedding's) vector representations and compare them.

Task:

Imagine you are developing a "**Personalized Reading Assistant**" for a startup that helps users find similar articles or reviews based on their interests. The assistant should take a user's input (e.g., a favorite review, article, or comment) and recommend the most relevant texts from a given dataset.

Dataset:

Use a dataset of **movie reviews**, **news articles**, or **product descriptions** (you can choose based on your interest).

Example: IMDB movie reviews, New York Times articles, or Amazon product reviews.

<https://www.kaggle.com/datasets/arashnic/mind-news-dataset>

Task 1: Data Preprocessing and Exploration [5 Marks]

a. Preprocess the text data to make it ready for vectorization:

- Remove URL's and special characters
- Lowercasing
- Stopword removal
- Stemming or Lemmatization
- Tokenization

b. Explore and visualize the data:

- Visualize the most frequent unigrams, bigrams, and trigrams.
- Show word distributions (e.g., using word clouds or bar plots).

Task 2: Implementing TF-IDF Based Similarity [4 Marks]

- a. Generate a TF-IDF matrix from the preprocessed data.
- b. Write a function to compute cosine similarity between a query (user input) and all documents in the dataset using TF-IDF vectors.
- c. Recommend the top 5 most similar texts based on cosine similarity.

Task 3: Implementing Word Embeddings for Similarity [6 Marks]

- a. Train a **Word2Vec model** on the dataset using both.
 - CBOW
 - Skip-gram
- b. Convert documents and the query input into **average word embeddings**.
- c. Compute cosine similarity between the query and documents using their word embeddings.
- d. Compare the top 5 recommendations from Word2Vec (CBOW and Skip-gram).

Task 4: Comparative Analysis [3 Marks]

- a. Compare the results of TF-IDF and Word2Vec-based recommendations:
 - Which method provided better recommendations? Why?
 - Discuss any limitations of each method.

You can download this notebook and add your code to complete the tasks:

<https://colab.research.google.com/drive/1X9Flz-sAHOwupd91DH9B0axzFehPhEQI?usp=sharing>

Task 5: Word Similarity [4 Marks]

Improve the similarity function to match the query entered by the user with a predefined list of bank names. An example is provided in the attached code (File name is “Task 5 code”).

Deliverables:

1. Code: Submit a well-documented Python script (.ipynb). The code should be clear and well-structured, with appropriate comments to enhance readability and understanding. Two separate notebooks (one for Tasks 1 to 4 and one for Task 5) should be submitted.
2. Report: Include:
 - Task 4 with proper explanations and visualizations.